1.1.1. Calculate Momentum

28:15

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:
m is the mass of the object (in kilograms).
v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases +

Explorer

calculate...

Submit

```
1  m=float(input())
2  v=float(input())
3  p=m * v
4  print(f"{p:.2f}kgm/s")
```

Debugger

Terminal Test cases

< Prev Reset Submit Next >

1.1.2. Conditional Calculation Based on the Number of Digits

08:54

Write a Python program that accepts an integer n as input. Depending on the number of digits in n.

Constraints:
$1 \leq n \leq 999$

Input Format:
- The input consists of a single integer n.

Output Format:
- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Sample Test Cases +

Explorer

condition...

Submit

```python
import math

# Accepting integer input from the user
try:
    line = input().strip()
    if line:
        n = int(line)

        # Determine the number of digits
        # Using string conversion is the most straightforward way
        num_digits = len(str(abs(n)))

        if 1 <= n <= 999:
            if num_digits == 1:
                # Single-digit: square the number
                print(n ** 2)

            elif num_digits == 2:
                # Two-digit: square root rounded to 2 decimal places
                result = math.sqrt(n)
                print(f"{result:.2f}")

            elif num_digits == 3:
                # Three-digit: cube root rounded to 2 decimal places
                # Using pow(n, 1/3) for cube root
```

Debugger

Terminal Test cases

< Prev Reset Submit Next >

1.1.3. Days between Two Dates

07:35

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format $YYYY-MM-DD$.
- The second line contains the second date in the format $YYYY-MM-DD$.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases +

Explorer

noOfDay...

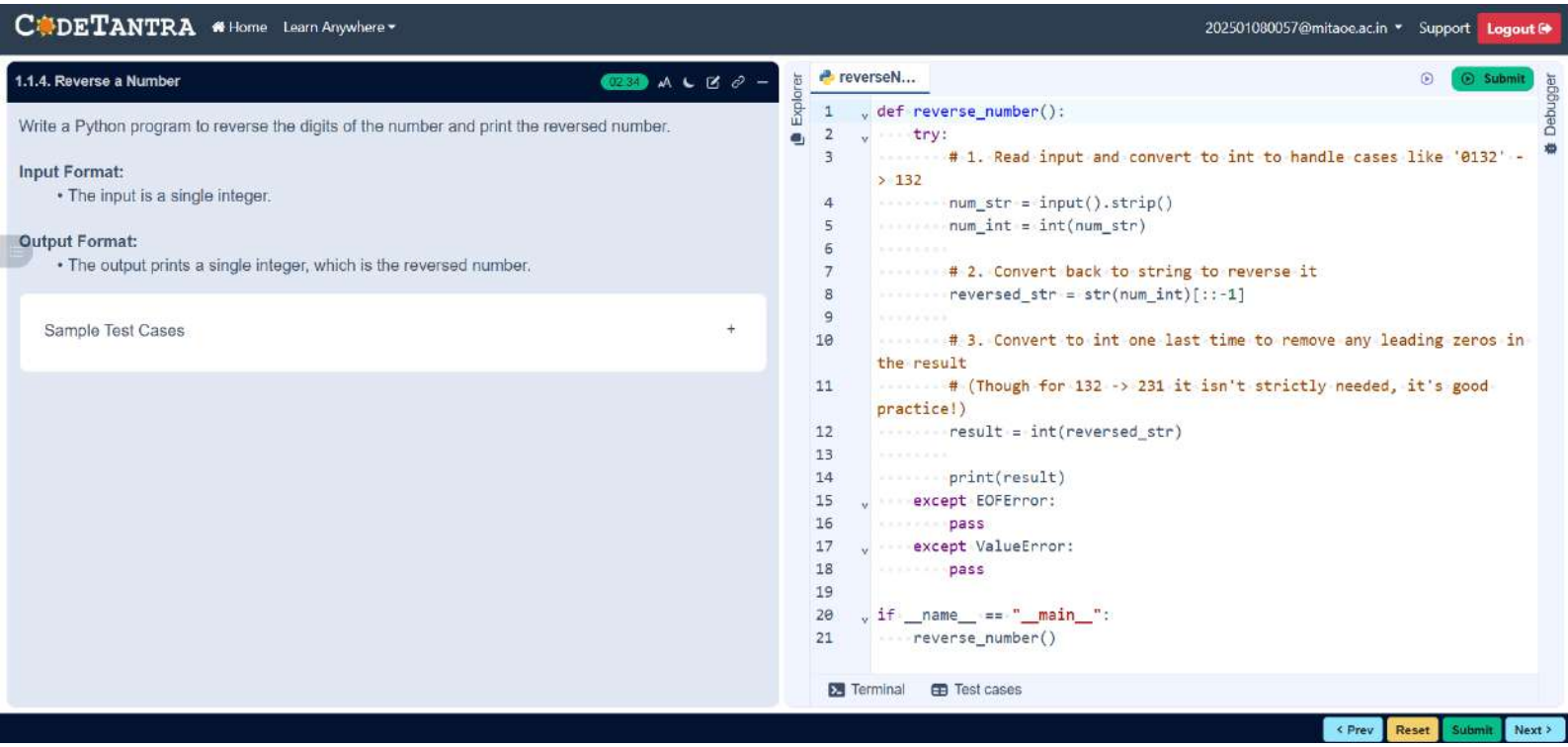
Submit

```python
from datetime import date
from datetime import datetime

def calculate_days():
    # Read the two date strings from input
    try:
        line1 = input().strip()
        line2 = input().strip()

        # Updated format to match "2014-07-02" (no spaces around hyphens)
        date_format = "%Y-%m-%d"

        # Convert strings to datetime objects
        date1 = datetime.strptime(line1, date_format)
        date2 = datetime.strptime(line2, date_format)

        # Calculate the difference
        delta = date2 - date1

        # Output only the integer (e.g., 123)
        print(delta.days)

    except EOFError:
        pass
    except Exception:
```

Terminal Test cases

Debugger

< Prev Reset Submit Next >


CODETANTRA
Home
Learn Anywhere
202501080057@mitaoe.ac.in
Support
Logout
1.1.4. Reverse a Number
02:34
Write a Python program to reverse the digits of the number and print the reversed number.
Input Format:
• The input is a single integer.
Output Format:
• The output prints a single integer, which is the reversed number.
Sample Test Cases
Explorer
reverseN...
Submit
Debugger
def reverse_number():
try:
# 1. Read input and convert to int to handle cases like '0132' -> 132
num_str = input().strip()
num_int = int(num_str)
# 2. Convert back to string to reverse it
reversed_str = str(num_int)[::-1]
# 3. Convert to int one last time to remove any leading zeros in the result
# (Though for 132 -> 231 it isn't strictly needed, it's good practice!)
result = int(reversed_str)
print(result)
except EOFError:
pass
except ValueError:
pass
if __name__ == "__main__":
reverse_number()
Terminal
Test cases
< Prev
Reset
Submit
Next >

1.1.5. Multiplication Table

00:59

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

```
<number> x <multiplier> = <result>
```

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Sample Test Cases +

Explorer

multiplica...

Submit

Debugger

```python
def print_multiplication_table():
    try:
        # Read the integer input
        n = int(input().strip())

        # Loop from 1 to 10 (range(1, 11) starts at 1 and stops before 11)
        for i in range(1, 11):
            # Calculate the result
            result = n * i

            # Print in the exact format: <number> x <multiplier> = <result>
            # We use an f-string for easy formatting
            print(f"{n} x {i} = {result}")

    except EOFError:
        pass
    except ValueError:
        pass

if __name__ == "__main__":
    print_multiplication_table()
```

Terminal Test cases

‹ Prev Reset Submit Next ›

1.2.1. Pass or Fail

03:01

Write a Python program that accepts the number of courses and the marks of a student in those courses.
The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n, the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:
If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses

Explorer

passorFa...

Submit

Debugger

```python
def calculate_grade():
    try:
        # Read the number of courses
        n_input = input().strip()
        if not n_input:
            return
        n = int(n_input)

        # Read the marks
        marks_input = input().strip()
        if not marks_input:
            return
        marks = list(map(int, marks_input.split()))

        # Check if any course is failed (marks < 40)
        has_failed = any(mark < 40 for mark in marks)

        if has_failed:
            print("Fail")
        else:
            # Calculate aggregate
            aggregate = sum(marks) / n

            # Print with the exact labels shown in the test case
            print(f"Aggregate Percentage: {aggregate:.2f}")
```

Terminal Test cases

< Prev Reset Submit Next >

1.2.3. Pattern - 1

04:36

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases +

Explorer

rightangl...

Submit

```
1  rows = int(input())
2
3  for i in range(1, rows + 1):
4      # Print the asterisk and a space repeated 'i' times
5      # This keeps the trailing space that the test case requires
6      print("* " * i)
```

Debugger

Terminal Test cases

< Prev Reset Submit Next >

1.2.4. Pattern - 2

01:39

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases +

Explorer

numberP...

Submit

Debugger

```python
# Read the number of rows
n = int(input())

# Outer loop for each row
for i in range(1, n + 1):
    # Inner loop to print numbers from 1 up to the current row number 'i'
    for j in range(1, i + 1):
        # Print the number followed by a space
        # end=" " keeps the output on the same line
        print(j, end=" ")

    # After the inner loop finishes, move to the next line
    print()
```

Terminal Test cases

2.1.1. List operations

05:16

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- **Add**: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display**: Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit**: Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice".Ensure that the program continues to prompt the user until they choose to quit (option 4).

Sample Test Cases +

Explorer

listOps.py

Submit

```
1  nums = []
2
3  while True:
4      print("1. Add")
5      print("2. Remove")
6      print("3. Display")
7      print("4. Quit")
8
9      choice = input("Enter choice: ")
10
11     if choice == '1':
12         try:
13             num = int(input("Integer: "))
14             nums.append(num)
15             print(f"List after adding: {nums}")
16         except:
17             print("Invalid input")
18
19     elif choice == '2':
20         if not nums:
21             print("List is empty")
22         else:
23             try:
24                 num = int(input("Integer: "))
25                 if num in nums:
```

Debugger

Terminal Test cases

< Prev Reset Submit Next >

2.2.1. Linear search Technique

00:28

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

Output format:

- If the element is found, print the index.
- If the element is not found, print **Not found**.

Sample Test Case:
Input:
1 2 3 4 3 5 6
3
Output:
2

Sample Test Cases +

Explorer

CTP1709...

Submit

```
# Input
arr = list(map(int, input().split()))
key = int(input())

# Linear Search
found = False
for i in range(len(arr)):
    if arr[i] == key:
        print(i)
        found = True
        break

# If not found
if not found:
    print("Not found")

```

Debugger

Terminal Test cases

< Prev Reset Submit Next >

2.2.2. Captain of the Team

00:26

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:
The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format
The output should be the height (in centimeters) of the tallest player.

Sample Test Cases +

Explorer

captainof...

Submit

```
1  # Input
2  heights = list(map(int, input().split()))
3
4  # Find tallest
5  tallest = max(heights)
6
7  # Output
8  print(tallest)
```

Debugger

Terminal Test cases